

KONGU ENGINEERING COLLEGE
(AUTONOMOUS)

PERUNDURAI, ERODE – 638 060

EXPERIMENT - 7

PROJECT TITLE

**SMART IRRIGATION AND WATER
MANAGEMENT SYSTEM**

SUBMITTED BY

ABISHEK R S	-	24MER002
GOWRISANKAR K	-	24MER010
NAVEENRAJ K	-	24MER035
PRAVEEN N M	-	24MER043

SMART IRRIGATION AND WATER MANAGEMENT SYSTEM

1. Project review:

The Smart Irrigation and Water Management System is a compact prototype designed to automatically monitor soil moisture and control water supply for agricultural applications. The system uses an ESP8266 NodeMCU as the central controller and a soil moisture sensor to detect the moisture content of the soil.

The soil moisture sensor continuously measures the soil condition and sends the sensor readings to the ESP8266 NodeMCU. The controller processes these values and compares them with a predefined moisture limit. When the soil becomes dry, the system automatically activates a relay to turn ON a water pump. When sufficient moisture is detected, the pump is turned OFF to avoid unnecessary water usage.

The proposed system provides a simple, low-cost, and automatic method of irrigation and water management. It can also be further developed with Wi-Fi connectivity, cloud monitoring, weather-based irrigation, mobile notifications, and data logging for a complete IoT-based smart agriculture system.

2. Problem Statement:

Traditional irrigation methods may result in excessive or insufficient water supply because irrigation is often controlled manually or according to a fixed schedule. This can lead to water wastage, poor crop growth, and inefficient use of resources.

Manual monitoring of soil moisture is inconvenient and does not provide continuous information about the actual water requirement of plants. Therefore, there is a need for a simple and low-cost system that can automatically monitor soil conditions and control irrigation when required.

The proposed system addresses these problems by integrating a soil moisture sensor, ESP8266 NodeMCU, relay module, water pump, buzzer, and LED. The soil moisture sensor detects the moisture level, while the ESP8266 processes the sensor readings. When the soil becomes dry, the system automatically activates the water pump. When sufficient moisture is reached, the pump is switched OFF.

3. Proposed Solution:

The proposed solution consists of an ESP8266-based Smart Irrigation and Water Management System. The soil moisture sensor is placed in the soil to continuously measure its moisture condition and sends the readings to the ESP8266 NodeMCU.

The ESP8266 processes the sensor readings and compares the measured moisture value with a predefined limit. If the soil moisture indicates a dry condition, the ESP8266 activates the relay to turn ON the water pump. An LED can provide a visual indication of the irrigation status, while a buzzer can provide an alert when irrigation is activated

Proposed Operation:

1. Start the system and initialize the ESP8266.
2. Initialize the soil moisture sensor, relay, buzzer, and LED.
3. The soil moisture sensor measures the moisture level.
4. The sensor sends the moisture data to the ESP8266.
5. ESP8266 processes the sensor reading.
6. The moisture value is compared with the predefined limit.
7. If the soil is dry, the relay activates the water pump.
8. The LED indicates the irrigation status.
9. The buzzer can provide an irrigation alert.
10. If sufficient moisture is detected, the water pump remains OFF.
11. The system continuously repeats the monitoring process.

4. System Architecture:

The Smart Irrigation and Water Management System consists of five main sections:

Input Section:

- **Soil Moisture Sensor** – Measures the moisture content of the soil.
- The sensor sends the measured data to the ESP8266 NodeMCU.

Processing Section:

The ESP8266 NodeMCU acts as the central controller.

- Receives soil moisture data.
- Processes the sensor readings.
- Compares the moisture value with the predefined limit.
- Controls the relay, water pump, buzzer, and LED.

Actuation Section:

The Relay Module can be used for machine protection.

- The relay receives a control signal from the ESP8266.
- It can be used to control a suitable machine load.
- It can demonstrate automatic shutdown during an abnormal condition.

Indication Section:

The Relay Module is used to control the water pump.

- The relay receives a control signal from the ESP8266.
- It switches the water pump ON when the soil is dry.
- It switches the water pump OFF when sufficient moisture is detected.

Connectivity Section:

The ESP8266 has built-in Wi-Fi capability.

The system can be further developed for:

- IoT-based irrigation monitoring.
- Remote pump control.
- Mobile notifications.
- Cloud data storage.
- Automatic water usage analysis.

5. Circuit Table:

Important Pin Mapping:

ESP8266 NodeMCU Pin Component

A0	A0 Soil Moisture Sensor → AO
GPIO 14	D5 Relay Module → IN
GPIO 12	D6 Buzzer → Positive Terminal
GPIO 13	D7 LED → Anode through resistor
3.3V / 5V	– Sensor Power
GND	– Common Ground

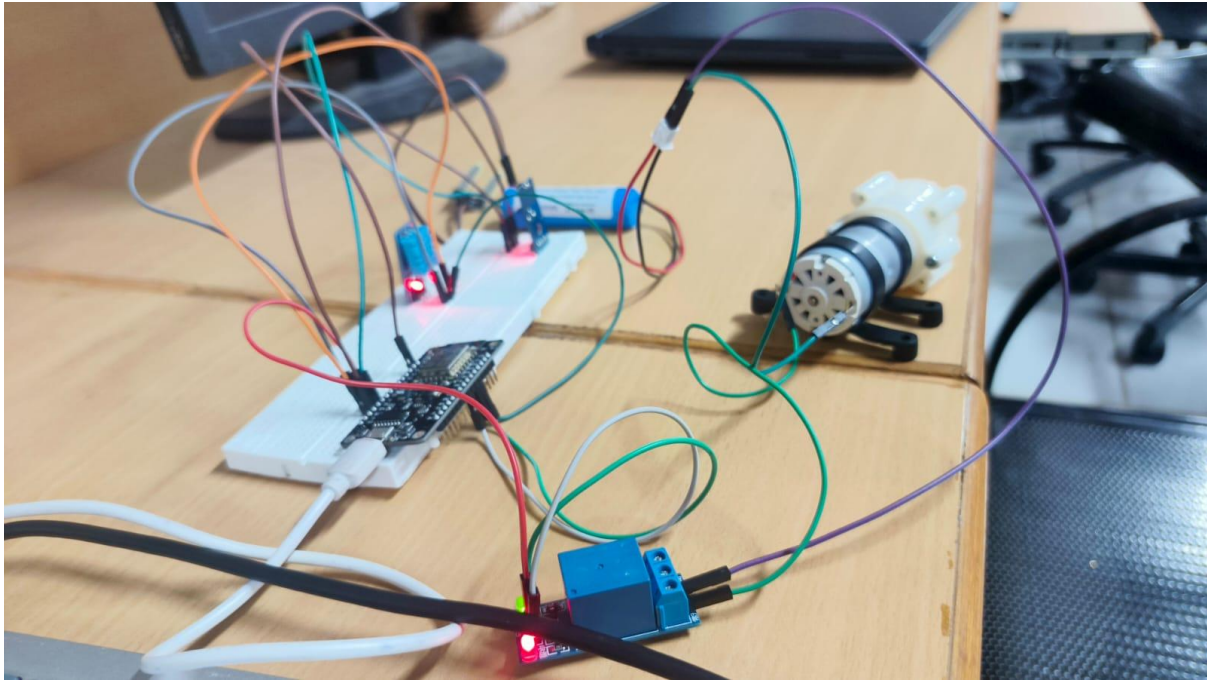
6. Circuit Design:

The circuit consists of an ESP8266 NodeMCU, soil moisture sensor, relay module, water pump, buzzer, LED, breadboard, and jumper wires.

The soil moisture sensor is placed in the soil and connected to the analog input A0 of the ESP8266. The sensor continuously measures the moisture condition of the soil.

The relay module is connected to GPIO14 (D5) and controls the water pump. The buzzer is connected to GPIO12 (D6) to provide an audible indication, while the LED is connected to GPIO13 (D7) through a suitable resistor to provide a visual indication.

The ESP8266 continuously monitors the soil moisture value. When the moisture level indicates dry soil, the relay is activated and the water pump is turned ON. When the soil reaches the required moisture level, the pump is automatically turned OFF.



7. Algorithm:

1. Start the system.
2. Initialize the ESP8266, temperature sensor, vibration sensor, buzzer, LED, and relay.
3. Read the temperature sensor value.
4. Read the vibration sensor value.
5. Compare the temperature with the predefined limit.
6. Check whether abnormal vibration is detected.
7. If an abnormal condition is detected, turn ON the buzzer, LED, and relay.
8. Display “MACHINE ALERT – MAINTENANCE REQUIRED.”
9. Otherwise, keep the buzzer and LED OFF and display “MACHINE HEALTH NORMAL.”
10. Wait for a short interval and repeat the monitoring process continuously.

8. Coding:

```
#include <ESP8266WiFi.h>
```

```
#include <ThingSpeak.h>
```

```
#include <DHT.h>
```

```
// =====
```

```
// SMART IRRIGATION AND WATER MANAGEMENT SYSTEM
```

```
// ESP8266 NodeMCU + Soil Moisture + DHT11 + Relay + Pump
```

```
// + WiFi + ThingSpeak
```

```
// =====
```

```
// ----- WIFI SETTINGS -----
```

```
const char* WIFI_SSID = "praveen";
```

```
const char* WIFI_PASSWORD = "praveen12345";
```

```
// ----- THINGSPEAK SETTINGS -----
```

```
// Your ThingSpeak Write API Key
```

```
const char* THINGSPEAK_API_KEY = "0IP4GOX0SMNK3JBV";
```

```
// IMPORTANT:
```

```
// Replace this with your ThingSpeak Channel ID
```

```
unsigned long CHANNEL_ID = 1234567;
```

```
// ----- PIN DEFINITIONS -----
```

```
#define SOIL_PIN A0

#define DHT_PIN 2    // D4 = GPIO2
#define RELAY_PIN 14  // D5 = GPIO14


#define DHTTYPE DHT11


// ----- DHT OBJECT -----

DHT dht(DHT_PIN, DHTTYPE);


// ----- WIFI OBJECT -----

WiFiClient client;


// ----- SETTINGS -----

// Adjust after checking your actual soil sensor readings
int dryThreshold = 700;


// Most relay modules are ACTIVE LOW
bool relayActiveLow = true;


// ThingSpeak update interval
unsigned long lastUpdate = 0;
const unsigned long UPDATE_INTERVAL = 20000;


// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  delay(1000);
```

```
  dht.begin();
```

```
  pinMode(RELAY_PIN, OUTPUT);
```

```
  // Pump OFF at startup
```

```
  if (relayActiveLow)
```

```
  {
```

```
    digitalWrite(RELAY_PIN, HIGH);
```

```
  }
```

```
  else
```

```
  {
```

```
    digitalWrite(RELAY_PIN, LOW);
```

```
  }
```

```
  Serial.println();
```

```
  Serial.println("=====");
```

```
  Serial.println(" SMART IRRIGATION SYSTEM");
```

```
  Serial.println(" ESP8266 NODEMCU");
```

```
Serial.println("=====");
```

```
// ----- WIFI CONNECTION -----
```

```
Serial.println();
```

```
Serial.print("Connecting to WiFi");
```

```
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
int attempts = 0;
```

```
while (WiFi.status() != WL_CONNECTED && attempts < 30)
```

```
{
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
    attempts++;
```

```
}
```

```
Serial.println();
```

```
if (WiFi.status() == WL_CONNECTED)
```

```
{
```

```
    Serial.println("WiFi connected!");
```

```
    Serial.print("IP Address: ");
```

```
    Serial.println(WiFi.localIP());
```

```
}
```

```
else
```

```
{  
  Serial.println("WiFi connection failed!");  
  Serial.println("Check WiFi name and password.");  
}
```

```
// ----- THINGSPEAK -----
```

```
ThingSpeak.begin(client);
```

```
Serial.println("ThingSpeak initialized.");
```

```
  delay(2000);  
}
```

```
// =====  
// RELAY CONTROL  
// =====
```

```
void pumpON()  
{  
  if (relayActiveLow)  
  {  
    digitalWrite(RELAY_PIN, LOW);  
  }  
  else  
  {  
    digitalWrite(RELAY_PIN, HIGH);  
  }  
}
```

```
}
```

```
Serial.println("PUMP STATUS    : ON");
```

```
}
```

```
void pumpOFF()
```

```
{
```

```
  if (relayActiveLow)
```

```
  {
```

```
    digitalWrite(RELAY_PIN, HIGH);
```

```
  }
```

```
  else
```

```
  {
```

```
    digitalWrite(RELAY_PIN, LOW);
```

```
  }
```

```
Serial.println("PUMP STATUS    : OFF");
```

```
}
```

```
// =====
```

```
// MAIN LOOP
```

```
// =====
```

```
void loop()
```

```
{
```

```
  // ----- READ SOIL MOISTURE -----
```

```
int soilValue = analogRead(SOIL_PIN);

// ----- READ DHT11 -----

float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

// ----- SERIAL OUTPUT -----

Serial.println();
Serial.println("-----");

Serial.print("Soil Moisture Value : ");
Serial.println(soilValue);

// Check DHT11 reading

if (isnan(temperature) || isnan(humidity))
{
    Serial.println("DHT11 ERROR!");
}
else
{
    Serial.print("Temperature      : ");
    Serial.print(temperature);
    Serial.println(" °C");
}
```

```
Serial.print("Humidity      : ");
Serial.print(humidity);
Serial.println(" %");
}

// ----- SOIL CONDITION -----

if (soilValue >= dryThreshold)
{
    Serial.println("SOIL CONDITION    : DRY");

    pumpON();

    Serial.println("ACTION          : Irrigation started");
}
else
{
    Serial.println("SOIL CONDITION    : WET");

    pumpOFF();

    Serial.println("ACTION          : Irrigation stopped");
}

// =====
// SEND DATA TO THINGSPEAK
// =====
```

```
if (WiFi.status() == WL_CONNECTED &&
    millis() - lastUpdate >= UPDATE_INTERVAL)
{
    lastUpdate = millis();

    Serial.println();
    Serial.println("Sending data to ThingSpeak...");

    // Field 1 = Soil Moisture
    ThingSpeak.setField(1, soilValue);

    // Field 2 = Temperature
    if (!isnan(temperature))
    {
        ThingSpeak.setField(2, temperature);
    }

    // Field 3 = Humidity
    if (!isnan(humidity))
    {
        ThingSpeak.setField(3, humidity);
    }

    // Field 4 = Pump Status
    if (soilValue >= dryThreshold)
    {
```

```
    ThingSpeak.setField(4, 1);
}
else
{
    ThingSpeak.setField(4, 0);
}

// Send data
int response = ThingSpeak.writeFields(
    CHANNEL_ID,
    THINGSPEAK_API_KEY
);

if (response == 200)
{
    Serial.println("ThingSpeak update successful!");
}
else
{
    Serial.print("ThingSpeak error code: ");
    Serial.println(response);
}
}

Serial.println("-----");
delay(3000);
}
```

Coding Output:

```
sketch_aug14b
{
  ThingSpeak.setField(3, humidity);
}

// Field 4 = Pump Status
if (soilValue >= dryThreshold)
{
  ThingSpeak.setField(4, 1);
}
else
{
  ThingSpeak.setField(4, 0);
}

// Send data
int response = ThingSpeak.writeFields(
  CHANNEL_ID,
  THINGSPEAK_API_KEY
);

if (response == 200)
{
  Serial.println("ThingSpeak update successful!");
}
else
{
  Serial.print("ThingSpeak error code: ");
  Serial.println(response);
}

Serial.println("-----");

delay(3000);
}
```

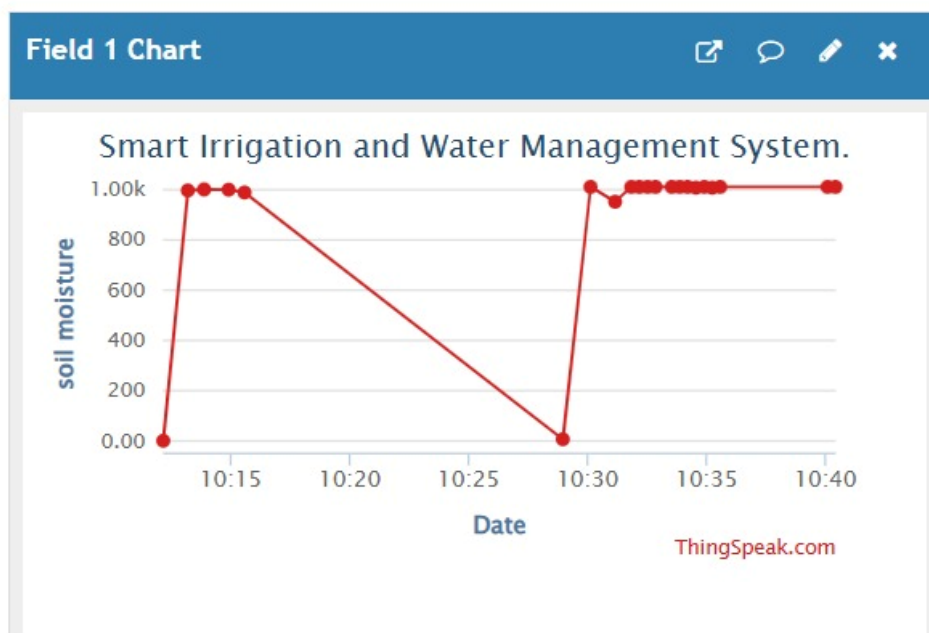
```
COM6
Sending data to ThingSpeak...
ThingSpeak update successful!
-----
Soil Moisture Value : 1011
Temperature       : 40.50 °C
Humidity          : 30.00 %
SOIL CONDITION    : DRY
PUMP STATUS       : ON
ACTION            : Irrigation started
-----
Autoscroll Show timestamp Hexline 9600 baud Clear output
```

Channel Stats

Created: 2 days ago

Last entry: less than a minute ago

Entries: 36



9. System Working:

Power ON the System:

The ESP8266 NodeMCU is powered ON.

Initialize Components:

The soil moisture sensor, relay module, buzzer, and LED are initialized.

Measure Soil Moisture:

The soil moisture sensor continuously measures the moisture content of the soil.

Send Sensor Data:

The sensors send the measured values to the ESP8266 NodeMCU.

Process the Data:

The ESP8266 receives and processes the sensor readings.

Determine Water Level:

The ESP8266 receives and processes the sensor reading.

Check Soil Condition:

The measured moisture value is compared with the predefined dry-soil limit.

Abnormal Machine Condition:

If excessive temperature or abnormal vibration is detected:

- Buzzer → ON
- LED → ON
- Relay → ON
- Machine alert → Activated
- Maintenance → Required

Normal Machine Condition:

If the water level is within the safe rangeIf the measured values are within the normal range:

- Buzzer → OFF
- LED → OFF
- Relay → OFF
- Machine condition → Normal

Repeat the Process:

- The sensors continuously monitor the machine, and the ESP8266 continuously checks the machine health.

10. Prototype Implementation:

The prototype is implemented using an ESP8266 NodeMCU, soil moisture sensor, relay module, water pump, buzzer, LED, breadboard, and jumper wires.

The soil moisture sensor is placed in the soil to detect its moisture content. The sensor sends the readings to the ESP8266, which automatically controls the water pump through the relay module.

Hardware Setup

1. Connect the Soil Moisture Sensor AO pin → A0 of ESP8266.
2. Connect the Relay IN pin → D5 (GPIO14).
3. Connect the Buzzer → D6 (GPIO12).
4. Connect the LED → D7 (GPIO13) through a suitable resistor.
5. Connect the soil moisture sensor to a suitable power supply.
6. Connect the relay module according to its required power supply.
7. Connect all GND terminals to a common ground.
8. Connect the water pump through the relay module.
9. Upload the program to the ESP8266 using the Arduino IDE.
10. Power ON the prototype.
11. Observe the soil moisture readings.
12. When the soil becomes dry, observe the automatic activation of the water pump.
13. Add water to the soil and observe the automatic stopping of irrigation.
14. The system continuously monitors and controls the irrigation process.

Prototype Operation:

The prototype demonstrates automatic machine condition monitoring and early warning. The sensors provide information about temperature and vibration

while the ESP8266 processes the readings and controls the buzzer, LED, and relay according to the programmed condition.

Prototype Working:

- The ESP8266 NodeMCU is powered ON and initializes all connected components.
- The soil moisture sensor continuously measures the soil condition.
- The sensor sends the measured value to the ESP8266.
- The ESP8266 processes the soil moisture reading.
- The measured value is compared with the predefined limit.
- If the soil is dry, the relay activates the water pump.
- The LED provides a visual indication of irrigation.
- The buzzer provides an audible alert.
- When sufficient moisture is detected, the relay switches OFF the water pump.
- The system continuously repeats the monitoring process.

11. Bill of Materials:

S.No.	Component	Specification	Quantity
1	ESP8266	NodeMCU Wi-Fi Development Board	1
2	Soil Moisture Sensor	Soil Moisture Detection Module	1
3	Relay Module	1-Channel Relay	1
4	Water Pump	Small DC Water Pump	1
5	Buzzer	3.3V/5V Buzzer	1
6	LED	Standard Indicator LED	1
7	Resistor	Suitable LED Current Limiting Resistor	1
8	Breadboard	Standard Prototype Board	1
9	Jumper Wires	Male-Male / Male-Female	1 Set
10	USB Cable	Micro-USB	1
11	Power Supply	Suitable Regulated Supply	1

12. Results & Performance Analysis:

- The soil moisture sensor successfully monitors the moisture condition of the soil.
- The ESP8266 NodeMCU successfully receives and processes the sensor readings.
- The system provides continuous monitoring of soil moisture.
- When the soil becomes dry, the relay activates the water pump.
- The LED provides a visual indication of irrigation.
- The buzzer provides an alert during irrigation activation.
- When sufficient moisture is reached, the water pump is automatically switched OFF.
- The system provides automatic and efficient water management

Results:

- The soil moisture sensor successfully detects the moisture condition of the soil.
- The ESP8266 NodeMCU successfully processes the sensor data.
- The system continuously monitors soil moisture.
- When dry soil is detected, irrigation is automatically activated.
- The relay successfully controls the water pump.
- The LED provides a visual indication.
- The system stops irrigation when sufficient moisture is detected.
- The prototype demonstrates the basic operation of a Smart Irrigation and Water Management System.

Performance Analysis:

Parameter	Expected Performance
Soil Moisture Monitoring	Continuous
Sensor Response	Real-time Reading
Dry Soil Detection	Automatic
Irrigation Control	Water Pump ON
Normal Soil Condition	Water Pump OFF
Visual Indication	LED ON During Irrigation
Water Management	Automatic and Efficient